

SQL CASE-STUDY PROJECT

Employee Performance Mapping

Designing a relational employee database and using SQL — joins, window functions, views, stored functions, and indexing — to map performance, reporting structure, and compensation across the organization.

Shivani Sharma

Data Analyst | Generative AI

PROJECT OVERVIEW

From Raw Records to Workforce Insight

One relational database, three linked tables, sixteen business questions answered in pure SQL.

OBJECTIVE

Build the Employee database from three source tables, then use SQL to profile employees, evaluate performance ratings, map reporting lines, standardize job levels, and benchmark pay — turning flat records into decisions a manager can act on.

DATABASE

Employee — MySQL relational database built with CREATE DATABASE / CREATE TABLE, populated from three CSV sources.



emp_record_table

Core employee master — role, department, salary, rating, manager, project



data_science_team

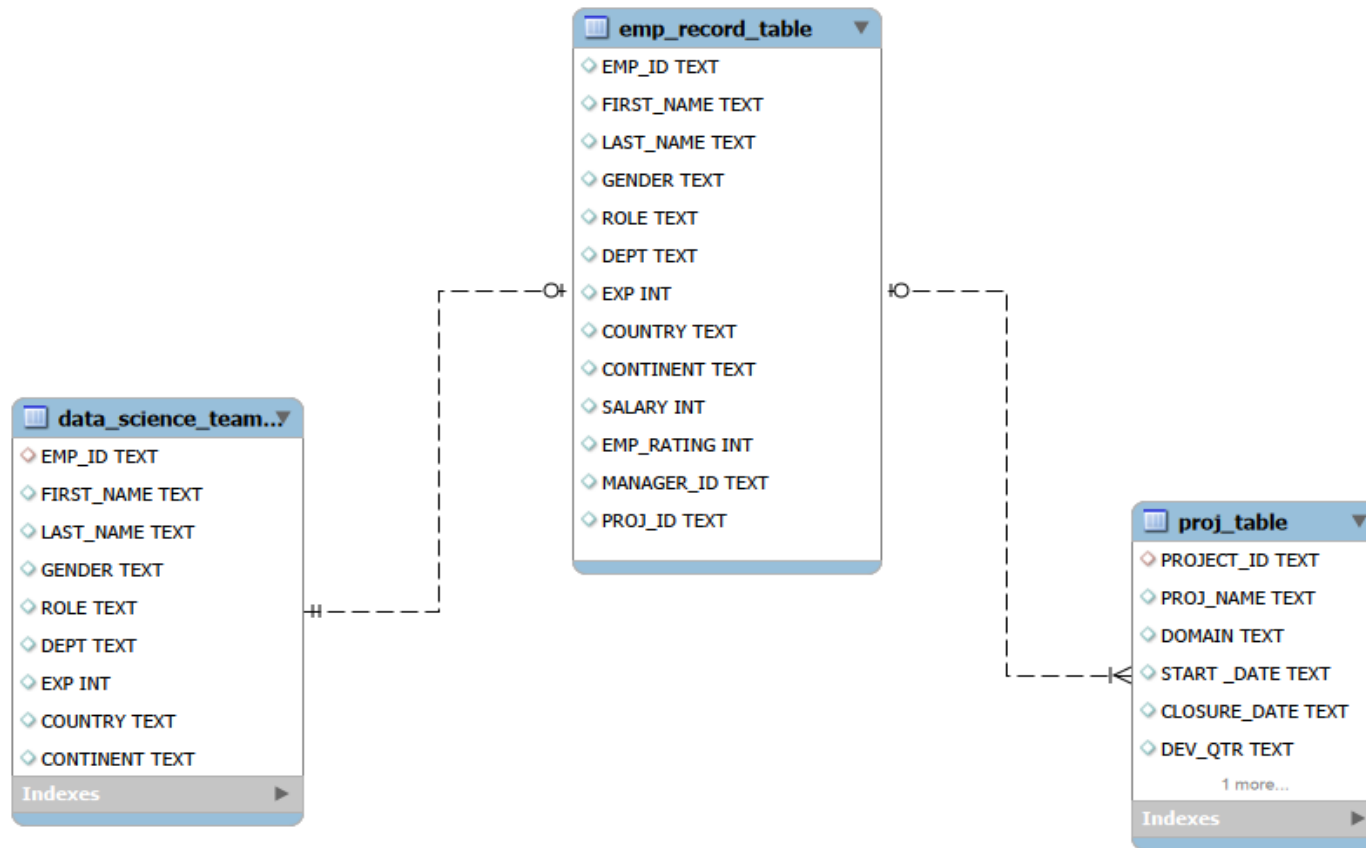
Data-science staff profiles used for job-level standardization



proj_table

Project catalog — domain, timeline, and delivery status

Entity Relationship Model



HOW THE TABLES CONNECT

- emp_record_table ↔ data_science_team**

Linked on EMP_ID — shared employee identity across the general workforce and the specialist team.

- emp_record_table ↔ proj_table**

Linked on PROJ_ID — every employee record maps to the project they are staffed on.

- Self-referencing MANAGER_ID**

EMP_ID on emp_record_table also appears as MANAGER_ID, enabling a self-join for reporting lines.

Filtering & Rating Segmentation

WHERE, BETWEEN, and string functions turn the employee master into targeted, decision-ready lists.

● **RATING < 2**

Flag employees needing performance support

● **RATING 2–4**

Core, steady-performing workforce segment

● **RATING > 4**

Top performers — candidates for recognition



Cross-department queries

EMP_ID / FIRST_NAME / LAST_NAME / GENDER / DEPT pulled for every employee, and department-specific views — e.g. Finance staff — built by combining CONCAT(FIRST_NAME, LAST_NAME) with a WHERE DEPT clause and an aliased NAME column.

Mapping Reporting Lines with a Self-Join

The employee table joined to itself to reveal who leads whom.



emp_record_table m
(the manager)

*m.EMP_ID =
e.MANAGER_ID*



emp_record_table e
(the reports)



Leadership identification

ROLE IN ('MANAGER', 'PRESIDENT', 'CEO') isolates every employee holding a leadership title, with their ID, name and department.



Combining departments with UNION

Healthcare and Finance employee lists are queried separately, then merged into one de-duplicated result set with UNION.

COUNT(e.EMP_ID) AS No_of_Reporters

Grouped by manager and ordered by team size, this surfaces span of control across the org — including the President — in a single query.

Window Functions for Row-Level Comparison

Ranking and benchmarking employees without collapsing the detail rows.



MAX() OVER (PARTITION BY DEPT)

Every employee row keeps its own EMP_RATING, alongside a new column showing the highest rating anywhere in that employee's department — an instant benchmark for how far each person sits from their department's best.

Partitioned, not grouped — detail rows survive alongside the aggregate.



RANK() OVER (ORDER BY EXP DESC)

Employees are ranked purely by years of experience, with tied experience levels receiving the same rank — producing a clean seniority ladder across the whole workforce in one pass.

Feeds directly into succession planning and promotion-readiness lists.

REUSABLE LOGIC

Views & Nested Queries

Packaging recurring logic so it can be reused — and asking questions that depend on their own answers.



employee_salary_view

CREATE VIEW

A saved query surfacing EMP_ID, name, country and salary for every employee earning above 6,000 — queried like a table, without rewriting the filter each time.

Use case: a live, always-current shortlist for compensation review — no duplicate logic to maintain.



Nested Subquery

WHERE EMP_ID IN (SELECT ...)

An inner query first isolates employees with more than ten years of experience; the outer query then pulls every column for exactly that set — answering a two-step question in one statement.

Use case: identifying senior, retention-critical talent for workforce planning.

AUTOMATION

Standardizing Job Levels with a Stored Function

A single `job_profile(exp_years)` function encodes the organization's title standard — and flags where reality drifts from it.



CASE WHEN ROLE = `job_profile(EXP)` THEN 'MATCH'

Applied across the `data_science_team` table, this compares each person's assigned ROLE to the title their experience should carry — a repeatable, single-query audit of title-to-experience alignment.

Optimizing Lookups with Indexing

Before adding an index, a name search means scanning every row in emp_record_table.

BEFORE

```
SELECT * FROM emp_record_table  
WHERE FIRST_NAME = 'Eric';
```

The execution plan shows a full table scan — MySQL checks every row because FIRST_NAME has no index to jump to.

AFTER

```
CREATE INDEX idx_first_name  
ON emp_record_table (FIRST_NAME(50));
```

The same query now uses the index to jump straight to matching rows — checked against the execution plan before and after to confirm the cost drop.

Bonus Calculation & Regional Benchmarking

Turning salary and rating fields into pay decisions and geographic comparisons.



Performance Bonus

$$\text{Bonus} = \text{Salary} \times 5\% \times \text{Employee Rating}$$

A single calculated column ties reward directly to performance — higher-rated employees on the same salary earn a proportionally larger bonus.



Salary by Continent & Country

AVG(SALARY) grouped by CONTINENT, COUNTRY

Aggregating pay across geography surfaces where compensation runs ahead of — or behind — the global average, informing pay-equity and location-strategy conversations.

SKILLS DEMONSTRATED

SQL Techniques Applied in This Project



Database & Schema Design

CREATE DATABASE, CREATE TABLE, ER modeling



Filtering & Conditionals

WHERE, BETWEEN, IN, string functions



Joins & Self-Joins

Reporting-line mapping within one table



Set Operations

UNION across department queries



Aggregate Functions

GROUP BY, MIN, MAX, AVG



Window Functions

PARTITION BY, RANK() OVER



Views

Reusable, always-current filtered datasets



Nested Subqueries

Queries built from queries



Stored Functions

Custom business-rule logic in SQL



Indexing & Optimization

Execution-plan-driven performance tuning

Thank You

Questions on the Employee Performance Mapping project are welcome.

Shivani Sharma

Data Analyst | Generative AI

shivani92754@gmail.com

+91 82945 99916

[linkedin.com/in/shivani-sharma-517636413](https://www.linkedin.com/in/shivani-sharma-517636413)